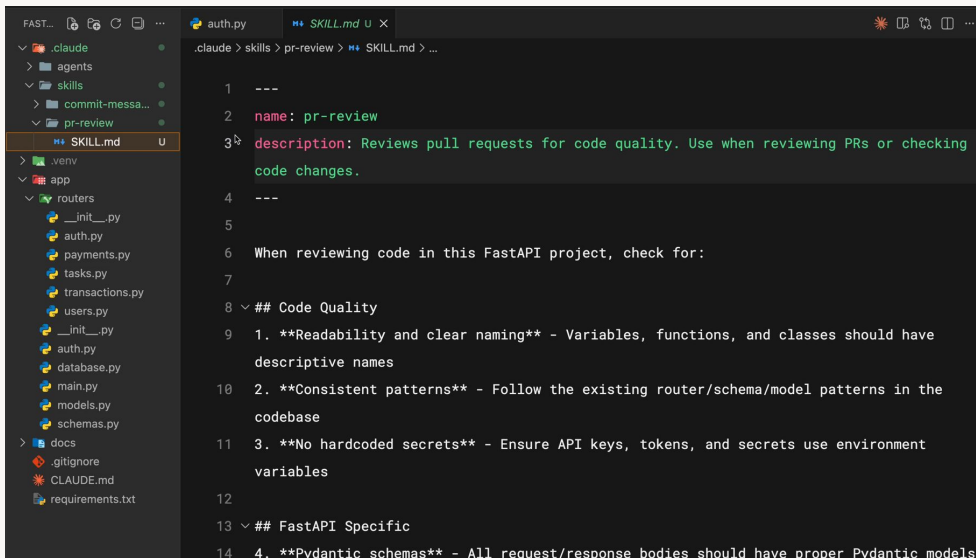


Introduction to Agent Skills

What are Skills?

What are Skills?



Here's what a skill's frontmatter looks like:

```
---
name: pr-review
description: Reviews pull requests for code quality. Use when reviewing
---
```

- Agent Skills are folders of instructions, scripts, and resources that Agents can discover and use to do things more efficiently & accurately
- With Claude Code, we have a SKILL markdown file that teaches Claude how to do something once and Claude applies that knowledge automatically when that task comes up
- Claude reads user requests, match it against available skill descriptions and activates the ones that match

Where Skills live?

- Each skill lives in a SKILL.md file with a name and description in its frontmatter
- Skills can be stored at different places based on who needs them
- Personal skills are stored in ~/.claude/skills and are applied for all your projects
- Personal skills can be used for user preferences like commit message style, documentation format, preferences for code explanation
- Project skills go in .claude/skills inside root directory of the project repository
- Project skills get committed to version control alongside your code, so the whole team shares them
- Project skills can be used for team standards live like company's brand guidelines, preferred fonts, colors used for web designing

What makes Skills different?

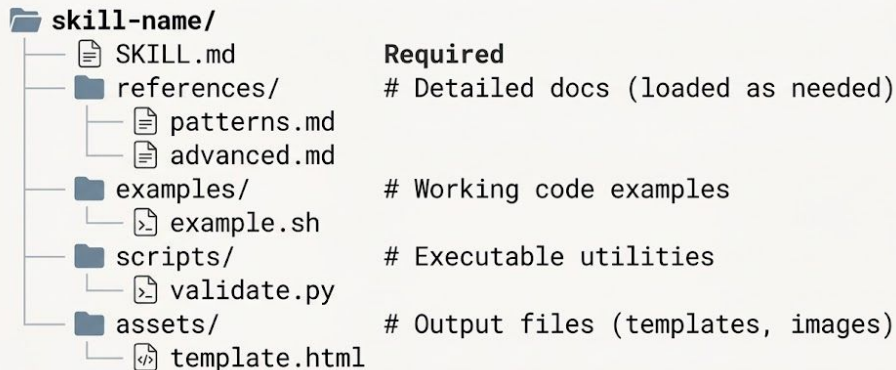
- Claude Code gives various options to customize its behaviour. Skills is unique as it's automatic and task-specific
- Claude only loads the name and description initially, so they don't fill up your entire context window
- How it's different from [claude.md](#) and /command:
 - [claude.md](#) files load into every conversation, whereas [skills.md](#) load on-demand when they match user request
 - Slash commands require explicit invocation whereas skills activate automatically (but can be invoked explicitly too)
- Example: PR review checklist doesn't need to be in the context window while debugging, but it loads when we ask for PR review

When to Use Skills?

- Skills work best for specialized knowledge that applies to specific tasks
- If you find yourself explaining the same thing to Claude repeatedly, that's a good use case for using skills
- Examples: teach Claude about team's coding standards, preferred format in commit message, brand guidelines of the organization

Creating Skills

Create a Personal Skill



- Create a directory for your skill with its name under `~/.claude/skills`
`mkdir -p ~/.claude/skills/<skill-name> && code ~/.claude/skills/<skill-name>`
- Define `SKILL.md`. Name identifies the skill and description is the matching criteria that tells Claude when to use it, along with the instructions to follow

name: create-pr

description: Create a complete PR workflow – branch, commit, push, and create PR

Argument hint: [branch-name-prefix]

<claude instructions to follow>

- Restart the session as Claude Code loads skills at startup and verify / test

How Claude Loads Skills?

- At startup, Claude scans 4 locations for and loads skill name and description
- Priority order for handling skill name conflict from highest to least:
 - `managed-settings.json`: managed settings inside enterprise paths
 - `~/.claude/skills/<skill-name>/SKILL.md`: personal skills inside home directory
 - `project/.claude/skills/<skill-name>/SKILL.md`: project skills inside project repository
 - `<plugin>/skills/<skill-name>/SKILL.md`: installed plugins that we get online
- Claude matches user request with skill descriptions with semantic matching
- Asks for user confirmation on loading up the skill which keeps user aware of the context Claude is using
- Claude then reads the complete skill markdown and follows its instructions

Multi-File Skills

Skill Metadata

Field	Required	Description
<code>name</code>	No	Display name for the skill. If omitted, uses the directory name. Lowercase letters, numbers, and hyphens only (max 64 characters).
<code>description</code>	Recommended	What the skill does and when to use it. Claude uses this to decide when to apply the skill. If omitted, uses the first paragraph of markdown content.
<code>argument-hint</code>	No	Hint shown during autocomplete to indicate expected arguments. Example: <code>[issue-number]</code> or <code>[filename] [format]</code> .
<code>disable-model-invocation</code>	No	Set to <code>true</code> to prevent Claude from automatically loading this skill. Use for workflows you want to trigger manually with <code>/name</code> . Default: <code>false</code> .
<code>user-invocable</code>	No	Set to <code>false</code> to hide from the <code>/</code> menu. Use for background knowledge users shouldn't invoke directly. Default: <code>true</code> .
<code>allowed-tools</code>	No	Tools Claude can use without asking permission when this skill is active.
<code>model</code>	No	Model to use when this skill is active.
<code>context</code>	No	Set to <code>fork</code> to run in a forked subagent context.
<code>agent</code>	No	Which subagent type to use when <code>context: fork</code> is set.
<code>hooks</code>	No	Hooks scoped to this skill's lifecycle. See Hooks in skills and agents for configuration format.

- The agent skills open standard supports several [metadata fields](#) in the skill markdown frontmatter that make them more effective
- *allowed-tools* restricts which tools Claude can use when the skill is active
- *model* specifies which Claude model to use for the skill

Best Practice: Skill Description

- Be clear and explicit with your description

Help me with docs ❌

Extracts text and tables from pdf files, fills forms and merges the documents provided. Use when working with pdf files or when the user mentions PDF forms or document extractions ✅

- A good description answers 2 questions:
 - What does the skill do?
 - When should Claude use it?
- If skill is not getting triggered reliably, add more trigger phrases and specific actions that match how users phrase their requests

Restricting Tool Use within Skills

- Sometimes we want the skill to only be able to read files, but not modify them – say for security sensitive workflows, read-only tasks, or any tasks where we need guardrails
- *allowed-tools* can be used to restrict the tool use within the scope of that skill
allowed-tools: Read, Grep, Glob
- While the skill is active, Claude can only use allowed-tools without asking permission
- If allowed-tools is omitted, the skill doesn't restrict anything. Claude uses its normal permission model

Best Practice: Progressive Disclosure

- Skills share Claude's context window with your conversation. When Claude activates a skill, it loads the contents of that SKILL.md into context
- Sometimes skills need references, examples, or utility scripts that the skill depends on
- Cramming all references, scripts, etc. into one [skill.md](#) eats away a lot of context window and is also hard to maintain
- Use [Progressive Disclosure](#) for skills > 500 lines by putting essential instructions in [skill.md](#) and detailed reference materials in separate files that Claude reads when needed
- Open Agent Skills standard suggests having below directories:
 - [scripts](#): executable code, e.g., Python, Bash, and JavaScript that agents can run
 - [References](#): additional focussed documentation, e.g., technical references, domain-specific files (legal, finance, ...) that agents can read when needed
 - [assets](#): for static resources like document and configuration templates, diagram examples, data files relevant for the skill

Best Practice: Progressive Disclosure

- General guidelines on how skills should be structured for efficient use of context:
 - Metadata (~100 tokens): The name and description fields are loaded at startup for all skills
 - Instructions (< 5000 tokens recommended): The full SKILL.md body is loaded when the skill is activated
 - Resources (as needed): Files (e.g. those in scripts/, references/, or assets/) are loaded only when required
- [SKILL.md](#) refer to these supporting resource files. For example, below is a snippet from [SKILL.md](#) and reference is loaded only when someone asks about system design
Only load when user requests more detail. See [architecture-guide.md](references/architecture-guide.md)
- This setup works similar to having a table of contents in the context window rather than fitting the entire document in there

Building Utility Scripts

- Scripts in skill directory can run without loading its contents into context window
- The key instruction to include in your SKILL.md is to tell Claude to run the script, not read it – only the script output will consume tokens
- Having utility scripts is useful for cases like:
 - Environment validation
 - Data transformations that need to be consistent
 - Operations that are more reliable as tested code than generated code

Claude Code Customization

Skills vs Claude.md

Skills	Claude.md
Load on-demand when matched with user request	Loads into every conversation always
Use for task-specific expertise and knowledge that's only relevant sometimes or detailed procedures that would clutter every conversation	Use for project wide standards that always apply
Examples: • Preferred format in the commit message • Brand guidelines of the organization	Examples: • Constraints like “never modify db schema” • Framework preferences and coding style

Skills vs Subagents

Skills	Subagents
Skills add knowledge to the current conversation. When skills activate, its instructions join the existing context	Subagents are isolated from the current conversation. They receive a task, work on it independently in a separate context, and return results
Use when: • you want to enhance Claude's knowledge for the current task • you want the expertise applied throughout the conversation	Use when: • you want to delegate a task to a separate execution context • you want isolation between delegated work and main context • different tool access than the main conversation is required

Skills vs Hooks

Skills	Hooks
Skills are request-driven and activate based on what the user is asking	Hooks fire-on events and are event driven
Use skills for: • Knowledge that informs how Claude handles requests • Guidelines that affect Claude's reasoning	Use hooks for: • Operations that run on every file save • Validation before specific tool call • Automated side effects of Claude's actions
	Examples: • hook might run a linter every time Claude saves a file • validate certain input before tool calls

Combining Claude.md, Skills, Subagents, Hooks

- A typical project may have:
 - **claude.md** for always-on project standards
Always use `pydantic` classes when defining new class
 - **Skills** for task-specific expertise
I just downloaded this repo, can you help me understand this codebase /codebase-onboarding
 - **Hooks** for automated operations
Auth before firing tool call to any API or database service
 - **MCP** to provide external tools and integrations
MCP tools to interact with slack and confluence content
- Each customization option handle its own specialty – combine them appropriately rather than forcing into one approach

Sharing Skills

Sharing Skills with Other Users

- Skills become more valuable when shared — PR review skill shared across the team standardizes code review and provides a consistent experience across the organization
- Ways to share your skills via:
 - Version Control: simplest sharing method is to commit skills to a repository and anyone who clones the repository gets these skills automatically
 - Works well for team coding standards and project specific workflows
 - Plugin Distribution: Plugins are way to extend Claude Code with custom functionality, but can be shared across teams and projects via marketplace.
Within <plugin-name> directory:
 - create plugin manifest <plugin-name>/claude-plugin/plugin.json
 - create skill within <plugin-name>/skills/<skill-name>/[SKILL.md](#)
 - Use to distribute general skills across repos and for broader community use

Sharing Skills with Other Users

```
C: > Users > lewis > managed-settings.json > {} strictKnownMarketplaces > {} 0 > ** repo
16  "env": {
17    "CLAUDE_CODE_ENABLE_TELEMETRY": "1",
18    "OTEL_METRICS_EXPORTER": "otlp"
19  },
20  "companyAnnouncements": [
21    "Welcome to Acme Corp! Review our code guidelines at docs.acme.com",
22    "Our new security policies are now active. For more info check out security.acme.com"
23  ],
24  "strictKnownMarketplaces": [
25    {
26      "source": "github",
27      "repo": "acme-corp/approved-plugins"
28    },
29    {
30      "source": "npm",
31      "package": "@acme-corp/compliance-plugins"
32    }
33  ]
34 }
```

- Enterprise Deployment: administrators can deploy skills organization-wide through managed settings
 - Enterprise skills take the highest priority and override personal, project, and plugin skills with the same name
 - Most appropriate for org-wide mandatory standards, security requirements, compliance workflows, and coding practices
 - Supports features like [strictKnownMarketplaces](#) to control where plugins can be installed from

Sharing Skills with Sub Agents

- Any delegated task to a sub agent starts with a fresh, clean context
- Sub agents don't inherit skills automatically:
 - Built-in skills like explorer, plan, and verify can't access skills at all
 - Only custom sub agents user defines can use skills, but ONLY when explicitly listed
 - Sub agent skills are loaded when the sub agent starts and not on-demand like in the main conversation
- To create a custom sub agent with skills:
 - create a new agent using claude code sub agent creator /agents > Create new agent
 - To update existing agent, update markdown within `.claude/agents/<agent-name.md>`
 - Add "skills" field in the agent's frontmatter to specify which skills to load

When to use Sub agents with Skills

```
---
name: frontend-security-accessibility-reviewer
description: "Use this agent when you need to review frontend code for a
tools: Bash, Glob, Grep, Read, WebFetch, WebSearch, Skill...
model: sonnet
color: blue
skills: accessibility-audit, performance-check
---
```

- For isolated task delegation with specific expertise and skillset
- To enforce standards and delegate work without relying on prompts
- Only list skills that are relevant for the sub agents task as they load by default on startup
- Different sub agents need different skills — frontend reviewer vs backend reviewer

Troubleshooting Skills

Common Skill Issues

- When skills don't work, the issues fall into one of these categories:
 - Skill doesn't load
 - Skill doesn't trigger
 - Skill has conflicts
 - Skill fails at runtime
- Start with Agent Skills Verifier:
 - Post creating a skill, try the [agent skills verifier](#) command – install using `uv sync`
 - Once installed, run `skills-ref validate <skill-path>` to validate for any structural issues before spending time on debugging other things

```
lewis > ...\agentskills\skills-ref > P main > 12:29 skills-ref validate C:\Users\lewis\fastapi-project\.claude\skills\codebase-onboarding\
Valid skill: C:\Users\lewis\fastapi-project\.claude\skills\codebase-onboarding
```

```
lewis > ...\agentskills\skills-ref > P main > 12:29
```

Skill Not Triggering

- If skill exists, passes agent skill validator checks, but Claude is not using it when expected, it's most likely due to poor description
- Define few concrete use cases before building to ensure trigger works for real user queries
- Claude uses semantic matching to match user requests with skill descriptions meaning
- Check your skill description against how user requests are phrased
- Add trigger phrases users would actually say in skill descriptions
Example: Use when user asks to "create a pull request" for a github repo
- Ensure trigger phrases do not overlap with other skills to avoid ambiguity
- Test with variations

Skill Not Loading

- If skill doesn't appear when Claude is asked what skills are available, check these things:
 - Is the skill in the right location with the right structure?
 - Is the [SKILL.md](#) file inside of a named directory and not skills root?
 - Is the file name exactly "[SKILL.md](#)"?
- Run `claude -- debug` to see loading errors and look for messages mentioning skill name
- If plugin skills are not appearing:
 - Clear cache with `rm ~/.claude/plugins/cache`
 - Restart Claude code and reinstall plugin
 - If skills still don't appear, it could be that the plugin structure might be wrong — use agent skills validator tool to check this

Skill Conflict

- If Claude uses the wrong skill or seems confused, check:
 - If multiple skills have too similar descriptions — make them distinct
 - Be as specific as possible — helps Claude decide when to use which skill while avoiding conflict with similar sounding skills
- Priority shadowing: If a skill is being ignored:
 - By default, priority order goes like: Enterprise > Personal > Project > Plugin
 - A higher priority skill like enterprise might have the same name as personal skill
 - Rename lower priority skill to something a bit more distinct

Skill Runtime Errors

- If skill loads but fails during execution:
 - Check for any external package dependency skill is using and install them
 - Add additional package dependencies in your skill descriptions
Example: Requires playwright to function properly
 - If skill is using a script, provide execute permissions
chmod +x ./check-setup.py
 - Check if the paths mentioned are correct with forward slashes, even on Windows

Troubleshooting Checklist

Skill Issue	Validate
Not Triggering	Improve skill descriptions and add trigger phrases
Not Loading	Check skill path, file name, and YAML syntax
Wrong Skill Used	Make skill descriptions more distinct
Skill Shadowing	Check skill priority and rename if needed
Plugin Skills Missing	Clear cache and reinstall plugin
Runtime Failures	Check dependencies, permissions, and paths